



Cryptographie

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction et principes fondamentaux

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Les objectifs de la cryptographie

La cryptographie moderne n'est pas qu'une technique de « secret » : c'est une discipline qui vise à garantir, en présence d'un adversaire potentiel capable d'observer, d'intercepter ou de modifier des communications, quatre propriétés fondamentales. Ces propriétés sont indépendantes les unes des autres — un système peut en garantir certaines sans garantir les autres — et un même mécanisme cryptographique ne les couvre pas toutes simultanément, d'où la nécessité de bien identifier l'objectif recherché avant de choisir une primitive.

- **Confidentialité** : seules les parties autorisées peuvent accéder au contenu en clair d'une information. C'est l'objectif historique de la cryptographie (le chiffrement), obtenu aujourd'hui par des algorithmes comme AES ou ChaCha20 (chapitre 2). La confidentialité ne dit rien sur le fait que le message ait pu être altéré en transit : un texte chiffré modifié peut, selon le mode utilisé, se déchiffrer en un texte clair corrompu sans que le récepteur ne s'en aperçoive.
- **Intégrité** : toute modification non autorisée d'une information — accidentelle (erreur de transmission) ou malveillante (altération délibérée) — doit être détectable par le récepteur. L'intégrité est assurée par des fonctions de hachage et des codes d'authentification de message (MAC/HMAC, chapitre 3), indépendamment de toute notion de confidentialité : un message peut être intègre sans être confidentiel (ex. une mise à jour logicielle publique, dont on vérifie l'empreinte).
- **Authenticité** : l'origine d'une information — l'identité de l'émetteur — est vérifiable par le récepteur. Un message authentique garantit qu'il provient bien de qui il prétend provenir, et pas d'un tiers qui usurperait cette identité. L'authenticité s'obtient via un MAC (authentification pour deux parties partageant un secret) ou une signature numérique (chapitre 5).
- **Non-répudiation** : l'émetteur d'un message ne peut pas nier ultérieurement, y compris devant un tiers indépendant (par exemple un juge), en être l'auteur. Cette propriété n'est apportée que par la signature numérique fondée sur la cryptographie asymétrique (chapitre 5) : un MAC symétrique, lui, ne l'apporte pas, puisque le récepteur possède la même clé que l'émetteur et pourrait techniquement avoir forgé le message lui-même.

En pratique, un système donné ne recherche pas toujours les quatre propriétés à la fois : un simple stockage de logs internes peut ne nécessiter que l'intégrité, tandis qu'un contrat électronique nécessitera authenticité et non-répudiation sans besoin de confidentialité.

2. Le principe de Kerckhoffs

Formulé par le cryptographe militaire Auguste Kerckhoffs en 1883, ce principe fondateur énonce que **la sécurité d'un système cryptographique ne doit reposer que sur le secret de la clé, jamais sur le secret de l'algorithme lui-même**. Autrement dit, on doit pouvoir publier intégralement la description d'un algorithme de chiffrement — son fonctionnement interne, son code source — sans que cela compromette la sécurité des messages chiffrés avec une clé restée secrète.

Un algorithme dont la sécurité dépend de rester confidentiel relève de la « sécurité par l'obscurité » (*security through obscurity*), une approche fondamentalement fragile pour au moins deux raisons : d'une part, il suffit que l'algorithme soit divulgué, rétro-ingénié ou simplement deviné une seule fois pour effondrer d'un coup la sécurité de tous les systèmes qui l'utilisent, sans possibilité de « changer de clé » pour se rétablir ; d'autre part, un algorithme tenu secret échappe à l'examen critique de la communauté scientifique, qui est historiquement le meilleur moyen de détecter ses faiblesses avant qu'un attaquant ne les exploite.

À l'inverse, un algorithme public, largement étudié, publié dans des conférences scientifiques et soumis à des années de cryptanalyse ouverte — comme AES ou RSA — gagne en confiance précisément *parce qu'il* résiste à un examen prolongé par des milliers de chercheurs indépendants. C'est cette logique qui explique pourquoi les standards actuels (AES, SHA-2/SHA-3, RSA, ECC) sont issus de processus de sélection publics (comme le concours AES du NIST) plutôt que de conceptions propriétaires gardées secrètes. Ce principe est le fil conducteur direct du module *Cryptanalyse*, qui étudie précisément comment ces algorithmes publics résistent — ou non — à l'analyse.

3. Confusion et diffusion (Claude Shannon, 1949)

Dans son article fondateur de 1949 sur la théorie de la communication des systèmes secrets, Claude Shannon a identifié deux propriétés structurantes que doit posséder tout chiffrement robuste pour résister à une analyse statistique du texte chiffré. Ces deux notions restent, plus de 75 ans plus tard, le cadre conceptuel de référence pour concevoir un chiffrement par bloc moderne.

- **Confusion** : rendre la relation entre la clé et le texte chiffré la plus complexe et la plus non linéaire possible, de sorte qu'observer le texte chiffré ne révèle statistiquement rien sur la clé utilisée. Concrètement, la confusion est apportée par des opérations de substitution non linéaires (les « boîtes-S », *S-boxes*), qui remplacent chaque symbole d'entrée par un autre symbole selon une table fixe conçue pour éviter toute relation algébrique simple.
- **Diffusion** : répartir l'influence de chaque bit du texte clair (et de chaque bit de la clé) sur l'ensemble des bits du texte chiffré, de sorte qu'une modification d'un seul bit du message clair change, en moyenne, environ la moitié des bits du texte chiffré produit. Ce phénomène est appelé l'**effet avalanche** : il empêche un attaquant de déduire des informations locales sur le clair à partir d'une observation localisée du chiffré. La diffusion est typiquement apportée par des opérations de permutation et de mélange linéaire qui répartissent chaque bit d'entrée sur de nombreuses positions de sortie.

Ces deux notions ne sont pas de simples curiosités théoriques : elles justifient directement la structure en « tours » (*rounds*) des chiffrements par blocs modernes, où chaque tour alterne une étape de substitution (confusion) et une étape de permutation/mélange (diffusion), répétées suffisamment de fois pour que l'effet cumulé rende toute analyse statistique du chiffré inexploitable. AES, étudié en détail au chapitre 2, est l'illustration directe de cette architecture confusion-diffusion.

4. Cryptographie symétrique vs asymétrique : vue d'ensemble

La cryptographie se divise en deux grandes familles, dont les propriétés sont largement complémentaires plutôt que concurrentes. Comprendre leurs forces et faiblesses respectives explique pourquoi les systèmes réels ne choisissent presque jamais l'une à l'exclusion de l'autre.

	Symétrique	Asymétrique
Clés	une seule clé secrète, partagée entre émetteur et récepteur	une paire de clés mathématiquement liées (publique/privée)
Vitesse	rapide (plusieurs Go/s en logiciel, davantage avec accélération matérielle)	beaucoup plus lente (plusieurs ordres de grandeur)
Problème principal	distribution sécurisée de la clé partagée à l'ensemble des correspondants	vérification de l'authenticité de la clé publique (qui la possède réellement ?)
Nombre de clés pour n correspondants	de l'ordre de n^2 si chaque paire a une clé distincte	n paires de clés, une par participant
Exemples	AES, ChaCha20	RSA, Diffie-Hellman, ECC

La cryptographie symétrique excelle pour chiffrer de gros volumes de données rapidement, mais pose un problème pratique redoutable : comment transmettre la clé secrète à l'autre partie sans qu'un tiers ne l'intercepte, alors que le canal de communication n'est justement pas encore sécurisé ? La cryptographie asymétrique résout élégamment ce problème d'amorçage (la clé publique peut être diffusée ouvertement), mais au prix d'une lenteur qui la rend impraticable pour chiffrer directement de grands volumes.

En pratique, les systèmes réels combinent systématiquement les deux approches : la cryptographie asymétrique sert à établir ou échanger une **clé de session** de façon sécurisée, laquelle est ensuite utilisée pour un chiffrement symétrique rapide de l'ensemble des données échangées. C'est le principe du **chiffrement hybride**, mis en œuvre notamment dans TLS (détaillé au chapitre 6) : l'asymétrique résout le problème de la distribution de clé, le symétrique assure le débit.

5. Notion de sécurité prouvable et de sécurité calculatoire

Contrairement à une intuition répandue, la sécurité d'un système cryptographique moderne n'est presque jamais une preuve mathématique d'impossibilité absolue : elle repose généralement sur une **hypothèse calculatoire**, c'est-à-dire l'existence d'un problème mathématique réputé difficile à résoudre en un temps raisonnable avec les moyens de calcul actuels et raisonnablement prévisibles dans un futur proche. Par exemple, la sécurité de RSA repose sur la difficulté supposée de factoriser un grand nombre entier produit de deux grands nombres premiers, et celle de Diffie-Hellman ou d'ECC repose sur la difficulté supposée du problème du logarithme discret. Ces problèmes ne sont pas démontrés impossibles à résoudre efficacement — ils sont seulement restés non résolus efficacement malgré des décennies d'efforts de recherche intensifs, ce qui constitue une forme de confiance empirique plutôt qu'une garantie absolue.

Cette distinction a une conséquence directe et importante : un progrès algorithmique inattendu (comme l'a démontré théoriquement l'algorithme de Shor pour un ordinateur quantique, cf. chapitre 6) peut potentiellement rendre soluble en pratique un problème jusque-là considéré comme difficile, invalidant du jour au lendemain la sécurité de tous les systèmes qui en dépendaient.

Il existe une exception notable et instructive : le **masque jetable** (*one-time pad*), seul système cryptographique dont la sécurité a été formellement démontrée par Shannon comme étant **inconditionnelle** (indépendante de toute puissance de calcul de l'attaquant, même infinie). Le masque jetable combine le message clair avec une clé aléatoire de même longueur par XOR, à condition impérative que la clé soit véritablement aléatoire, utilisée une seule fois, et aussi longue que le message. Ces contraintes le rendent impraticable pour un usage courant (il faudrait distribuer et gérer une clé aussi volumineuse que l'ensemble des données à protéger), ce qui explique pourquoi la cryptographie moderne repose presque exclusivement sur la sécurité calculatoire plutôt que sur cette sécurité prouvable inconditionnelle.

6. Vocabulaire de base

Avant d'aborder les mécanismes concrets dans les chapitres suivants, il est utile de fixer un vocabulaire précis, employé de façon cohérente dans tout le module :

Terme	Définition
Texte clair (plaintext)	le message original, lisible, non protégé, avant toute transformation cryptographique
Texte chiffré (ciphertext)	le résultat du chiffrement du texte clair, en principe inintelligible sans la clé appropriée
Clé	le paramètre secret (ou la paire publique/privée) qui contrôle le fonctionnement précis du chiffrement et du déchiffrement
Algorithme (chiffre, cipher)	la procédure de transformation elle-même, publique par principe (Kerckhoffs), qui définit <i>comment</i> la clé agit sur le texte
Cryptosystème	l'ensemble complet formé par un algorithme, sa gestion des clés (génération, distribution, stockage, révocation) et le protocole qui encadre son usage réel

Cette distinction entre « algorithme » et « cryptosystème » est importante : un algorithme mathématiquement solide (AES, par exemple) peut être intégré dans un cryptosystème vulnérable si la gestion des clés est défectueuse — clé codée en dur, générateur aléatoire de mauvaise qualité, absence de rotation des clés. La majorité des failles cryptographiques rencontrées en pratique proviennent d'ailleurs de la gestion des clés et du protocole d'usage, bien plus que d'une faiblesse de l'algorithme sous-jacent lui-même.

À retenir

- Confidentialité, intégrité, authenticité, non-répudiation : quatre objectifs distincts, pas toujours tous nécessaires simultanément selon le contexte, et pas tous apportés par les mêmes mécanismes (un MAC n'apporte pas la non-répudiation, contrairement à une signature).
- Le principe de Kerckhoffs interdit de fonder la sécurité sur le secret de l'algorithme ; seule la clé doit rester secrète, l'algorithme doit pouvoir être publié et étudié ouvertement.
- Confusion (relation clé/chiffré non linéaire) et diffusion (effet avalanche) sont les deux propriétés de Shannon qui structurent la conception des chiffrements par blocs modernes.
- La cryptographie moderne combine en pratique symétrique (rapide, adapté au volume) et asymétrique (échange de clé, authentification) dans des schémas hybrides, chacune compensant la faiblesse de l'autre.
- La sécurité cryptographique moderne repose sur des hypothèses calculatoires (problèmes réputés difficiles), non sur une preuve d'impossibilité absolue — à l'exception du masque jetable, prouvé mais impraticable.

Chapitre 2 — Cryptographie symétrique : chiffrement par bloc et par flux

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Chiffrement par flux (stream cipher)

Un chiffrement par flux combine le texte clair, bit par bit ou octet par octet, avec une suite pseudo-aléatoire appelée le **keystream**, générée à partir de la clé (et généralement d'un nonce). L'opération de combinaison la plus courante est le XOR, réversible par construction :

```
chiffré[i] = clair[i] XOR keystream[i]
déchiffré[i] = chiffré[i] XOR keystream[i] (identique, car XOR est son propre inverse)
```

L'avantage principal d'un chiffrement par flux est sa légèreté : il ne nécessite pas de découper le message en blocs de taille fixe ni de gérer de padding, et il peut chiffrer un flux de données de longueur arbitraire, potentiellement inconnue à l'avance. Exemple moderne largement utilisé : **ChaCha20**, conçu par Daniel J. Bernstein, apprécié pour sa rapidité en logiciel (y compris sur des processeurs sans accélération matérielle dédiée, contrairement à AES) et sa résistance aux attaques par canal auxiliaire liées au temps d'exécution.

Un chiffrement par flux exige impérativement de **ne jamais réutiliser le même couple (clé, nonce)** pour chiffrer deux messages différents. En effet, si le même keystream sert à chiffrer deux messages clairs distincts `m1` et `m2`, un attaquant qui observe les deux textes chiffrés `c1` et `c2` peut calculer :

```
c1 XOR c2 = (m1 XOR keystream) XOR (m2 XOR keystream) = m1 XOR m2
```

Le keystream s'annule et l'attaquant obtient directement le XOR des deux messages clairs — une fuite d'information souvent suffisante pour reconstituer une grande partie du contenu, en particulier si l'un des deux messages est partiellement connu ou prévisible (rappel du module *Cryptanalyse*).

2. Chiffrement par bloc (block cipher)

Un chiffrement par bloc, à la différence d'un chiffrement par flux, chiffre des blocs de taille fixe (128 bits pour AES) en appliquant plusieurs « tours » (*rounds*) de transformations combinant substitution et permutation, conformément aux principes de confusion et de diffusion vus au chapitre 1.

AES (Advanced Encryption Standard)

AES a été standardisé par le NIST en 2001 à l'issue d'un concours public international, à partir d'un algorithme belge nommé Rijndael (conçu par Joan Daemen et Vincent Rijmen). Ce processus de sélection ouvert illustre directement le principe de Kerckhoffs : plusieurs candidats ont été soumis à une cryptanalyse publique intensive avant qu'un vainqueur ne soit désigné. AES opère sur des blocs de 128 bits avec des clés de 128, 192 ou 256 bits, appliquées en 10, 12 ou 14 tours respectivement — plus la clé est longue, plus le nombre de tours augmente, pour maintenir une marge de sécurité comparable.

Chaque tour comprend classiquement quatre opérations, appliquées dans cet ordre :

1. **SubBytes** : substitution non linéaire octet par octet à l'aide d'une boîte-S fixe et publique, assurant la **confusion** (aucune relation algébrique simple entre l'octet d'entrée et l'octet de sortie).
2. **ShiftRows** : décalage cyclique des lignes de la matrice d'état représentant le bloc, contribuant à la **diffusion** en déplaçant les octets les uns par rapport aux autres.
3. **MixColumns** : combinaison linéaire des octets de chaque colonne de l'état (opération absente au tout dernier tour), qui renforce fortement la diffusion en mélangeant les octets d'une même colonne entre eux.
4. **AddRoundKey** : XOR de l'état courant avec une sous-clé de tour, dérivée de la clé principale par un algorithme d'expansion de clé (*key schedule*) — c'est la seule étape qui introduit réellement le secret dans le calcul.

La répétition de ce cycle sur plusieurs tours garantit qu'un changement d'un seul bit de la clé ou du clair affecte, en sortie, la quasi-totalité des bits du bloc chiffré (effet avalanche).

3. Modes opératoires : pourquoi ils sont indispensables

Un chiffrement par bloc, pris isolément, ne sait traiter qu'un seul bloc de taille fixe : il faut donc un **mode opératoire** pour définir comment enchaîner le chiffrement d'une succession de blocs afin de traiter un message de longueur arbitraire, généralement supérieure à 128 bits. Le choix du mode a un impact de sécurité au moins aussi important que le choix de l'algorithme sous-jacent lui-même.

Mode	Principe	Points clés
ECB (Electronic Codebook)	chaque bloc clair est chiffré indépendamment des autres, avec la même clé	à proscrire absolument : deux blocs clairs identiques produisent deux blocs chiffrés identiques, ce qui laisse apparaître des motifs (structure d'une image, répétitions d'un fichier) directement dans le texte chiffré, sans même avoir besoin de casser l'algorithme
CBC (Cipher Block Chaining)	chaque bloc clair est XORé avec le bloc chiffré précédent avant d'être chiffré ; nécessite un vecteur d'initialisation (IV) aléatoire et imprévisible pour le premier bloc	corrige le défaut de motifs de l'ECB, mais vulnérable aux attaques de padding oracle si l'application signale (par un code d'erreur distinct ou un délai de réponse différent) que le padding du message déchiffré est invalide
CTR (Counter)	transforme un chiffrement par bloc en un chiffrement par flux, en chiffrant un compteur incrémental (souvent nonce + numéro de bloc) puis en XORant le résultat avec le clair	parallélisable (chaque bloc peut être chiffré indépendamment, contrairement à CBC qui est séquentiel) et permet un accès direct à n'importe quel bloc sans déchiffrer les précédents ; mais un nonce/compteur réutilisé avec la même clé est aussi dangereux qu'en chiffrement par flux pur
GCM (Galois/Counter Mode)	mode CTR combiné à une authentification intégrée reposant sur une multiplication dans un corps de Galois (mode AEAD)	recommandé par défaut dans la quasi-totalité des déploiements modernes : fournit confidentialité ET intégrité/authenticité en une seule primitive, tout en restant parallélisable comme CTR

Le choix entre ces modes n'est jamais anodin : ECB doit être systématiquement rejeté dès la conception, y compris pour des cas d'usage qui paraissent anodins, car la découverte de motifs peut suffire à un attaquant pour déduire des informations sensibles sans même effectuer de cryptanalyse au sens strict.

4. AEAD : chiffrement authentifié

Un mode **AEAD** (*Authenticated Encryption with Associated Data*), comme AES-GCM ou ChaCha20-Poly1305, combine en une seule opération le chiffrement du message et le calcul d'un code d'authentification, produisant un **tag d'authentification** (typiquement 128 bits) qui est vérifié lors du déchiffrement. Le nom complet souligne une capacité supplémentaire utile en pratique : les données associées (*associated data* — par exemple un en-tête de paquet réseau, ou des métadonnées non chiffrées) peuvent être authentifiées sans être chiffrées, ce qui est fréquent dans les protocoles réseau où certains champs doivent rester lisibles par des équipements intermédiaires tout en étant protégés contre toute altération.

Si le texte chiffré (ou les données associées) a été modifié — que ce soit accidentellement, par une corruption de transmission, ou délibérément par un attaquant actif — la vérification du tag échoue et le déchiffrement est **explicitement rejeté**, plutôt que de produire silencieusement un texte clair corrompu ou manipulé que l'application pourrait traiter à tort comme légitime. Cette propriété est essentielle : sans elle, un attaquant capable de modifier un texte chiffré en transit pourrait, dans certains cas, manipuler le comportement de l'application réceptrice sans jamais avoir besoin de casser le chiffrement lui-même.

Recommandation pratique actuelle : privilégier systématiquement un mode AEAD (AES-GCM, ChaCha20-Poly1305) plutôt que de composer soi-même un mode CBC avec un HMAC calculé séparément (chapitre 3). Une composition manuelle de ce type est une source fréquente d'erreurs d'implémentation — mauvais ordre entre chiffrement et authentification, réutilisation involontaire de clé entre les deux opérations, oubli de vérifier le tag avant de traiter le clair — alors qu'un mode AEAD intégré élimine ces classes d'erreurs par construction.

5. Gestion de la clé et du vecteur d'initialisation (IV)/nonce

La robustesse mathématique d'un algorithme comme AES ne protège en rien contre une mauvaise gestion opérationnelle de ses paramètres, qui reste l'une des causes les plus fréquentes de compromission en pratique.

- La **clé** doit impérativement être générée par un générateur de nombres aléatoires cryptographiquement sûr (CSPRNG — *Cryptographically Secure Pseudo-Random Number Generator*), et jamais codée en dur dans le code source, ni dérivée d'une source prévisible (horodatage, identifiant de processus). Une clé codée en dur, retrouvée par simple lecture du code (y compris décompilé) ou fuitée via un dépôt de code source, réduit à néant toute la sécurité apportée par l'algorithme.
- L'**IV/nonce** doit être unique pour chaque opération de chiffrement effectuée avec une même clé : aléatoire et imprévisible pour CBC, souvent un simple compteur incrémental pour CTR/GCM (l'unicité important davantage que l'imprévisibilité dans ce dernier cas). Sa réutilisation avec la même clé est l'une des erreurs d'implémentation les plus dommageables en pratique : pour CBC, elle facilite certaines analyses ; pour CTR et GCM, elle expose directement le XOR des messages clairs comme pour un chiffrement par flux, et pour GCM spécifiquement, la réutilisation d'un nonce peut permettre à un attaquant de forger des tags d'authentification valides, brisant l'intégrité en plus de la confidentialité.

6. Dérivation de clé à partir d'un mot de passe

Un mot de passe choisi par un être humain — même perçu comme « fort » par son auteur — n'a généralement pas l'entropie (l'imprévisibilité mesurable) suffisante pour servir directement de clé de chiffrement de 128 ou 256 bits : l'espace des mots de passe réellement utilisés par les humains est très inférieur à l'espace théorique des clés de cette taille, et un attaquant peut exploiter cet écart par une attaque par dictionnaire ou par force brute ciblée.

C'est pourquoi on ne dérive jamais une clé de chiffrement directement d'un mot de passe par une simple fonction de hachage générique : on utilise une **fonction de dérivation de clé** (KDF — *Key Derivation Function*) dédiée à cet usage, comme **PBKDF2**, **scrypt** ou **Argon2**. Ces fonctions sont conçues pour être volontairement coûteuses en temps de calcul et, pour scrypt et Argon2, également coûteuses en mémoire — un choix délibéré qui ralentit fortement un attaquant tentant de tester un grand nombre de mots de passe candidats (attaque hors ligne sur une base de mots de passe volée, par exemple), sans pour autant pénaliser de façon perceptible un utilisateur légitime qui ne dérive une clé qu'une seule fois. Ce mécanisme rejoint directement le hachage sécurisé des mots de passe abordé au chapitre 3 et approfondi dans le module *Cryptanalyse*.

À retenir

- AES est le standard de chiffrement par bloc de référence (128 bits de bloc, clés de 128/192/256 bits) ; sa sécurité effective dépend fortement du mode opératoire choisi, pas seulement de l'algorithme lui-même.
- ECB est à proscrire en toute circonstance car il laisse transparaître des motifs du clair ; les modes AEAD (GCM, ChaCha20-Poly1305) sont recommandés par défaut car ils apportent confidentialité et intégrité simultanément et éliminent les erreurs de composition manuelle.
- La réutilisation d'un nonce/IV avec la même clé est l'une des erreurs d'implémentation les plus graves en cryptographie symétrique, pouvant exposer le clair et, pour GCM, permettre la forge de tags d'authentification.
- Un mot de passe humain ne doit jamais servir directement de clé : une fonction de dérivation dédiée (PBKDF2, scrypt, Argon2), volontairement coûteuse, doit s'interposer.

Chapitre 3 — Fonctions de hachage et authentification de message (MAC/HMAC)

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

Une fonction de hachage H transforme une entrée de taille arbitraire — un mot, un fichier de plusieurs gigaoctets, un flux de données — en une sortie de taille fixe, appelée **empreinte** ou **digest** (par exemple 256 bits pour SHA-256, quelle que soit la taille de l'entrée). Pour être utilisable en cryptographie, une telle fonction doit satisfaire trois propriétés de sécurité, dont les attaques correspondantes sont détaillées côté cryptanalyse au chapitre 4 du module *Cryptanalyse* :

- **Résistance à la préimage** : étant donné une empreinte h , il doit être calculatoirement infaisable de retrouver une entrée m telle que $H(m) = h$. Cette propriété est ce qui rend un hachage de mot de passe utile : connaître l'empreinte stockée ne doit pas permettre de retrouver directement le mot de passe.
- **Résistance à la seconde préimage** : étant donné une entrée m_1 , il doit être infaisable de trouver une entrée différente $m_2 \neq m_1$ telle que $H(m_1) = H(m_2)$.
- **Résistance aux collisions** : il doit être infaisable de trouver, sans contrainte sur les entrées, une paire quelconque (m_1, m_2) avec $m_1 \neq m_2$ telle que $H(m_1) = H(m_2)$. Cette propriété est plus forte et plus difficile à garantir que les deux précédentes (le paradoxe des anniversaires implique qu'une collision devient statistiquement probable après environ $2^{(n/2)}$ essais pour une sortie de n bits, contre 2^n pour une préimage).

Une fonction de hachage cryptographique n'a par ailleurs aucune clé : à la différence d'un chiffrement, H est une fonction publique et déterministe — la même entrée produit toujours la même sortie, pour n'importe qui.

Panorama des fonctions usuelles

Fonction	Taille de sortie	Statut
MD5	128 bits	cassée (collisions pratiques démontrées dès les années 2000), à ne plus jamais utiliser pour un usage sécuritaire, même si elle subsiste dans des contextes non sécuritaires (sommes de contrôle d'intégrité accidentelle)
SHA-1	160 bits	cassée (collision pratique démontrée publiquement en 2017), dépréciée par tous les référentiels de sécurité actuels
SHA-2 (SHA-256, SHA-384, SHA-512)	256 à 512 bits	recommandée, aucune attaque pratique connue à ce jour ; construction de type Merkle-Damgård, comme MD5 et SHA-1, mais avec une fonction de compression bien plus robuste
SHA-3 (Keccak)	224 à 512 bits	recommandée, construction interne radicalement différente de SHA-2 (fonction éponge plutôt que Merkle-Damgård), ce qui diversifie le risque : une faille structurelle qui affecterait un jour la famille SHA-2 n'affecterait pas nécessairement SHA-3, et inversement

Le remplacement de MD5 puis de SHA-1 illustre un phénomène récurrent en cryptographie : une fonction jugée sûre à une époque donnée peut, avec l'avancée de la cryptanalyse et de la puissance de calcul disponible, cesser de l'être — d'où l'importance de suivre les recommandations à

2. Usages des fonctions de hachage

Les fonctions de hachage sont l'une des briques les plus universellement utilisées de la cryptographie, bien au-delà du seul chiffrement :

- **Vérification d'intégrité** : comparer l'empreinte d'un fichier téléchargé à une valeur de référence publiée par la source de confiance (souvent affichée à côté du lien de téléchargement) ; toute altération du fichier, même d'un seul bit, produit une empreinte totalement différente et donc détectable.
- **Structures de données** : les **arbres de Merkle**, où chaque nœud interne est le hachage de ses enfants, permettent de vérifier efficacement l'intégrité d'un très grand ensemble de données sans devoir tout retélécharger (utilisés en blockchain, module suivant, et dans les systèmes de fichiers distribués) ; les **tables de hachage**, structure de données classique en informatique, pour un usage non nécessairement cryptographique.
- **Brique de construction** : dérivation de clé (KDF), génération de nombres pseudo-aléatoires, signatures numériques où l'on signe l'empreinte d'un message plutôt que le message entier (chapitre 5), preuve de travail (*proof of work*) en blockchain.
- **Hachage de mots de passe** : nécessite impérativement des fonctions dédiées et volontairement lentes — **bcrypt**, **scrypt**, **Argon2** — jamais une fonction générique rapide comme SHA-256 utilisée seule, qui permettrait à un attaquant disposant du matériel adapté de tester des milliards de mots de passe candidats par seconde (rappel du module *Cryptanalyse*, chapitre 4). La lenteur volontaire de ces fonctions est un choix de conception délibéré, symétrique à celui des KDF pour la dérivation de clé vu au chapitre 2.

3. Code d'authentification de message (MAC)

Un MAC (*Message Authentication Code*) garantit simultanément **l'intégrité** d'un message et **l'authenticité** de son origine, pour deux parties qui partagent au préalable une clé secrète commune : $MAC = F_k(\text{message})$, où seule une partie possédant la clé k peut produire un MAC valide, ou vérifier qu'un MAC reçu correspond effectivement au message et à la clé.

Concrètement, l'émetteur calcule le MAC du message avec la clé partagée et l'envoie accolé au message ; le récepteur recalcule le MAC de son côté avec la même clé et vérifie qu'il correspond à celui reçu. Toute modification du message en transit — par un attaquant actif ou par une erreur de transmission — produit un MAC recalculé différent de celui reçu, révélant l'altération.

À la différence d'une signature numérique fondée sur la cryptographie asymétrique (chapitre 5), un MAC **ne fournit pas la non-répudiation** : toute partie capable de vérifier un MAC est, par construction, également capable d'en produire un valide, puisqu'elle possède exactement la même clé secrète que l'émetteur. Un MAC prouve donc « ce message a été produit par quelqu'un connaissant la clé partagée », mais ne permet pas de distinguer, entre les deux parties qui partagent cette clé, laquelle a effectivement produit le message — une distinction sans importance dans un échange bilatéral de confiance, mais rédhibitoire dès qu'il faut convaincre un tiers.

4. HMAC (Hash-based MAC)

HMAC est une construction générique standardisée permettant de bâtir un MAC robuste à partir de n'importe quelle fonction de hachage sous-jacente (HMAC-SHA256, HMAC-SHA512, etc.) :

$$\text{HMAC}(k, m) = H((k' \text{ XOR } \text{opad}) || H((k' \text{ XOR } \text{ipad}) || m))$$

où k' désigne la clé complétée (par des zéros si besoin) jusqu'à la taille de bloc interne de H , $||$ la concaténation, et ipad/opad deux constantes fixes et distinctes, définies dans le standard, de la taille d'un bloc de H .

Cette construction en **double application** de H , avec la clé combinée différemment de part et d'autre (via ipad puis opad), n'est pas un simple artifice : elle neutralise spécifiquement l'**attaque par extension de longueur** qui rend dangereuse une construction naïve du type $H(\text{clé} || \text{message})$ avec une fonction de hachage de type Merkle-Damgård (comme SHA-256). Dans une telle construction naïve, un attaquant qui connaît $H(\text{clé} || \text{message})$ peut, sans connaître la clé, calculer $H(\text{clé} || \text{message} || \text{données_ajoutées})$ pour des données de son choix appendues à la fin — une faille structurelle étudiée en détail au chapitre 4 du module *Cryptanalyse*. HMAC échappe entièrement à cette faille grâce à sa structure imbriquée en deux passes.

HMAC-SHA256 est aujourd'hui l'un des mécanismes de MAC les plus utilisés en pratique : authentification de requêtes d'API (signature de requêtes AWS, par exemple), jetons **JWT** signés avec l'algorithme **HS256**, intégrité de sessions applicatives, authentification de messages dans de nombreux protocoles réseau.

5. MAC intégré : GMAC et Poly1305

Les modes AEAD vus au chapitre précédent (AES-GCM, ChaCha20-Poly1305) n'ont pas besoin de HMAC : ils intègrent directement un mécanisme de MAC spécifique — **GMAC** pour AES-GCM (fondé sur une multiplication dans un corps de Galois), **Poly1305** pour ChaCha20 (fondé sur l'évaluation d'un polynôme modulo un grand nombre premier) — dans l'opération de chiffrement elle-même. Ce couplage évite d'avoir à composer manuellement une opération de chiffrement et une opération de MAC distinctes.

Cette composition manuelle, lorsqu'elle est malgré tout nécessaire (parce qu'un mode AEAD n'est pas disponible dans l'environnement cible, par exemple), est une source d'erreurs classique, en particulier lorsque l'ordre entre chiffrement et authentification est mal choisi (authentifier le message avant de le chiffrer plutôt que l'inverse, voir section suivante) — un piège fréquent qui a historiquement conduit à des protocoles vulnérables, alors qu'un mode AEAD intégré élimine cette classe d'erreurs par construction en imposant le bon ordre en interne.

6. Ordre chiffrement/authentification (Encrypt-then-MAC)

Lorsqu'un schéma AEAD intégré n'est pas utilisé et qu'il faut composer soi-même un chiffrement et un MAC, trois ordres sont possibles, et ils ne sont pas équivalents en sécurité :

- **Encrypt-then-MAC** (chiffrer d'abord, puis calculer le MAC sur le texte chiffré) : approche **recommandée**. Le récepteur vérifie d'abord le MAC sur le texte chiffré reçu, et rejette immédiatement tout message dont le MAC est invalide, sans jamais tenter de le déchiffrer. Cela limite fortement la surface d'attaques par oracle (notamment le padding oracle sur CBC) : un attaquant qui modifie le texte chiffré est détecté avant que le système ne révèle la moindre information sur le résultat d'un déchiffrement tenté.
- **MAC-then-Encrypt** : le MAC est calculé sur le clair puis l'ensemble (message + MAC) est chiffré. Le récepteur doit déchiffrer avant de pouvoir vérifier le MAC, ce qui expose potentiellement le système à des attaques exploitant le comportement du déchiffrement d'un texte chiffré invalide (attaques de type padding oracle notamment).
- **Encrypt-and-MAC** : chiffrement et MAC sont calculés indépendamment sur le clair. Outre une exposition similaire à MAC-then-Encrypt, le MAC lui-même peut, selon la fonction utilisée, fuiter de l'information sur le clair puisqu'il porte directement sur celui-ci.

Ce choix rejoint directement la conception des modes AEAD : GCM et ChaCha20-Poly1305 appliquent en interne une logique équivalente à Encrypt-then-MAC, ce qui est une des raisons pour lesquelles ils sont recommandés par défaut plutôt qu'une composition manuelle.

À retenir

- Une fonction de hachage cryptographique doit résister à la préimage, à la seconde préimage et aux collisions ; MD5 et SHA-1 sont cassées et dépréciées, SHA-2 et SHA-3 sont recommandées.
- Un MAC garantit intégrité et authenticité pour des parties partageant une clé secrète, mais sans non-répudiation, contrairement à une signature numérique.
- HMAC neutralise spécifiquement l'attaque par extension de longueur inhérente aux fonctions de hachage basées sur Merkle-Damgård, via sa construction en double application imbriquée.
- Privilégier un mode AEAD intégré (GCM, Poly1305) plutôt que de composer manuellement chiffrement et MAC ; si une composition manuelle est nécessaire, appliquer impérativement le principe Encrypt-then-MAC.

Chapitre 4 — Cryptographie asymétrique : RSA, Diffie-Hellman, ECC

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe général

Dans un système de cryptographie asymétrique (dite aussi cryptographie à clé publique), chaque partie dispose d'une **paire de clés mathématiquement liées** : une **clé publique**, qui peut être diffusée librement et sans restriction à n'importe qui, et une **clé privée**, qui doit rester strictement secrète et n'est jamais partagée. La relation mathématique entre les deux clés est conçue pour qu'il soit calculatoirement infaisable de retrouver la clé privée à partir de la seule clé publique, malgré leur lien direct.

Cette paire de clés permet deux usages principaux, qui exploitent la relation dans des sens opposés :

- **Chiffrement** : n'importe qui peut chiffrer un message avec la clé **publique** du destinataire ; seul le détenteur de la clé **privée** correspondante peut le déchiffrer. C'est le mécanisme utilisé pour garantir la confidentialité vis-à-vis d'un destinataire précis.
- **Signature** : le détenteur de la clé **privée** signe un message ; n'importe qui peut vérifier cette signature avec la clé **publique** correspondante, sans jamais avoir accès à la clé privée (chapitre 5). C'est le mécanisme utilisé pour garantir l'authenticité et la non-répudiation.

Cette dissymétrie — qui peut chiffrer/vérifier contre qui peut déchiffrer/signer — est ce qui distingue fondamentalement la cryptographie asymétrique de la cryptographie symétrique vue au chapitre 2, où une seule et même clé sert aux deux opérations inverses.

2. Rappels d'arithmétique modulaire nécessaires

La cryptographie asymétrique classique (RSA, Diffie-Hellman, ECC) repose sur des notions d'arithmétique modulaire qu'il est utile de rappeler avant d'aborder la construction des algorithmes :

- **Congruence modulo n** : deux entiers a et b sont dits congrus modulo n , noté $a \equiv b \pmod{n}$, s'ils ont le même reste dans la division euclidienne par n . L'arithmétique modulaire se comporte comme une horloge qui « boucle » après n : c'est ce bouclage qui rend les problèmes sous-jacents difficiles à inverser.
- **Exponentiation modulaire** : le calcul de $a^e \pmod{n}$ est au cœur de RSA et de Diffie-Hellman. Il serait impraticable de calculer d'abord a^e en entier (un nombre pouvant compter des milliers de chiffres) puis de le réduire modulo n ; on utilise donc l'algorithme d'**exponentiation rapide** (« square and multiply »), qui calcule le résultat en un nombre d'opérations proportionnel au nombre de bits de l'exposant e , jamais à sa valeur.
- **Petit théorème de Fermat / théorème d'Euler** : ces résultats classiques de théorie des nombres, portant sur le comportement des puissances modulo un nombre premier (Fermat) ou modulo un entier quelconque (Euler, via l'indicatrice d'Euler $\varphi(n)$), constituent le fondement mathématique qui garantit que le déchiffrement RSA « annule » correctement le chiffrement.
- **Inverse modulaire** : trouver d tel que $e \times d \equiv 1 \pmod{\varphi(n)}$ — c'est-à-dire l'inverse de e dans l'arithmétique modulo $\varphi(n)$ — se calcule efficacement grâce à l'**algorithme d'Euclide étendu**. C'est précisément ainsi que la clé privée RSA d est dérivée de la clé publique e et des deux facteurs premiers p et q , gardés secrets.

3. RSA : construction

RSA (du nom de ses concepteurs Rivest, Shamir et Adleman, 1977) reste l'algorithme asymétrique le plus connu. Sa construction se déroule en cinq étapes :

1. Choisir deux grands nombres premiers **distincts** p et q (typiquement de 1024 à 2048 bits chacun pour une sécurité actuelle) ; calculer $n = p \times q$, appelé le **module RSA**.
2. Calculer $\varphi(n) = (p-1)(q-1)$, l'indicatrice d'Euler de n .
3. Choisir un exposant public e , premier avec $\varphi(n)$ — en pratique presque systématiquement la valeur **65537** ($2^{16} + 1$), choisie car elle rend l'exponentiation rapide tout en évitant certaines faiblesses connues d'exposants trop petits (comme $e = 3$).
4. Calculer l'exposant privé d , l'inverse modulaire de e modulo $\varphi(n)$ (via Euclide étendu, cf. section précédente).
5. La **clé publique** est la paire (n, e) ; la **clé privée** est la paire (n, d) . Les nombres premiers p et q eux-mêmes doivent être détruits ou conservés secrets — ils ne font pas directement partie des clés mais leur connaissance permettrait de recalculer d .

Chiffrement : $c = m^e \bmod n$. **Déchiffrement** : $m = c^d \bmod n$. La correction de ce mécanisme (le fait que déchiffrer redonne bien le message original) découle directement du théorème d'Euler mentionné plus haut.

La sécurité de RSA repose sur la difficulté, pour un attaquant qui ne connaît que la clé publique (n, e) , de retrouver d — ce qui, en l'état actuel des connaissances, revient à devoir **factoriser** n en ses deux facteurs premiers p et q . Ce problème de factorisation de grands entiers est réputé difficile pour des tailles de module suffisamment grandes avec les algorithmes classiques connus (rappel : chapitre 5 du module *Cryptanalyse*).

Padding : une nécessité, pas une option

RSA « brut » (appliqué directement sur le message, sans traitement préalable) est **déterministe** — chiffrer deux fois le même message avec la même clé publique donne toujours le même texte chiffré — et vulnérable à plusieurs attaques structurelles bien documentées (attaques par exploitation de la structure multiplicative de RSA, attaques sur de petits exposants, entre autres — cf. module *Cryptanalyse*). Les standards actuels imposent donc systématiquement un schéma de **padding** avant l'exponentiation modulaire : **OAEP** (*Optimal Asymmetric Encryption Padding*) pour le chiffrement, **PSS** (*Probabilistic Signature Scheme*) pour la signature (détaillé au chapitre 5). Utiliser RSA sans un padding correctement implémenté est une erreur d'implémentation grave, quelle que soit par ailleurs la taille de la clé choisie — une clé de 4096 bits mal utilisée sans padding reste vulnérable.

4. Échange de clé Diffie-Hellman

L'échange de clé **Diffie-Hellman** (Whitfield Diffie et Martin Hellman, 1976 — la première publication de cryptographie à clé publique) permet à deux parties d'établir un **secret partagé** sur un canal de communication public et non sécurisé, sans jamais transmettre ce secret directement sur le canal :

1. Les deux parties se mettent d'accord sur des paramètres publics : un grand nombre premier p et un générateur g (ces valeurs peuvent être connues de tous, y compris d'un attaquant).
2. Alice choisit un secret aléatoire a , calcule $A = g^a \bmod p$ et l'envoie à Bob.
3. Bob choisit un secret aléatoire b , calcule $B = g^b \bmod p$ et l'envoie à Alice.
4. Alice calcule $B^a \bmod p$, Bob calcule $A^b \bmod p$: les deux calculs aboutissent mathématiquement à la même valeur $g^{(ab)} \bmod p$, qui devient le secret partagé, jamais transmis en tant que tel sur le canal.

Un observateur qui intercepte l'intégralité du trafic voit p , g , A et B , mais retrouver le secret partagé $g^{(ab)} \bmod p$ à partir de ces seules valeurs publiques (sans connaître a ni b) requiert de résoudre le **problème du logarithme discret** — retrouver a à partir de g et $A = g^a \bmod p$ — réputé difficile pour des paramètres suffisamment grands, avec les mêmes garanties calculatoires (et non prouvées) que pour la factorisation RSA.

Limite importante : Diffie-Hellman « pur », tel que décrit ci-dessus, ne fournit **aucune authentification** des parties. Un attaquant positionné entre Alice et Bob (attaque de l'**homme du milieu**, *man-in-the-middle*) peut mener un échange Diffie-Hellman distinct avec chacune des deux parties, se faisant passer auprès de chacune pour l'autre, sans que ni Alice ni Bob ne s'en aperçoivent — l'attaquant déchiffre alors et rechiffre tout le trafic en toute transparence. Diffie-Hellman doit donc impérativement être combiné à un mécanisme d'authentification indépendant (certificats, signatures — chapitre 5) pour être utilisable en pratique, ce qui est précisément le rôle joué par les certificats de serveur dans TLS (chapitre 6).

5. Cryptographie sur courbes elliptiques (ECC)

La cryptographie sur courbes elliptiques remplace le groupe multiplicatif modulo p utilisé par Diffie-Hellman classique par le **groupe des points d'une courbe elliptique** définie sur un corps fini — un objet géométrique et algébrique différent, mais qui admet une opération de « multiplication par un scalaire » jouant un rôle analogue à l'exponentiation modulaire. Le problème équivalent au logarithme discret dans ce contexte, appelé **ECDLP** (*Elliptic Curve Discrete Logarithm Problem*), est considéré nettement plus difficile à résoudre à taille de clé égale que le logarithme discret classique.

Cette difficulté accrue par bit de clé permet d'utiliser des clés **bien plus courtes** pour un niveau de sécurité équivalent : une règle de correspondance approximative, rappelée au chapitre 5 du module *Cryptanalyse*, situe une clé ECC de 256 bits à un niveau de sécurité comparable à une clé RSA de 3072 bits. Des clés plus courtes signifient des calculs plus rapides, des échanges réseau plus légers et une empreinte de stockage réduite — un avantage particulièrement appréciable sur des appareils contraints (mobile, IoT) ou pour des protocoles à fort volume d'échanges comme TLS.

Les courbes les plus utilisées aujourd'hui sont **P-256** (standardisée par le NIST, très répandue dans les certificats TLS existants) et **Curve25519** (conçue par Daniel J. Bernstein, utilisée par **X25519** pour l'échange de clé et **Ed25519** pour la signature — chapitre 5), cette dernière étant privilégiée dans les déploiements modernes pour sa résistance aux erreurs d'implémentation courantes et l'absence de certains paramètres controversés présents dans les courbes du NIST.

6. Chiffrement hybride

En pratique, la cryptographie asymétrique n'est presque jamais utilisée pour chiffrer directement de gros volumes de données : les opérations asymétriques (exponentiation modulaire pour RSA, multiplication scalaire sur courbe pour ECC) sont plusieurs ordres de grandeur plus lentes que les opérations symétriques équivalentes (chapitre 2), ce qui les rend impraticables au-delà de très petites quantités de données.

Elle sert donc plutôt à **établir ou transmettre une clé de session** de taille fixe (par exemple 256 bits pour AES-256), qui est ensuite utilisée avec un chiffrement symétrique rapide (typiquement AES-GCM) pour protéger l'ensemble du volume de données réellement échangées. C'est le principe du **chiffrement hybride**, déjà introduit au chapitre 1, et c'est exactement le mécanisme mis en œuvre par TLS (chapitre 6) : un échange de clé asymétrique (souvent ECDHE, Diffie-Hellman éphémère sur courbe elliptique) établit un secret partagé, dont dérivent les clés symétriques qui chiffrent réellement tout le trafic applicatif de la session.

À retenir

- RSA repose sur la difficulté de factoriser un grand entier produit de deux nombres premiers ; Diffie-Hellman et ECC reposent sur la difficulté du problème du logarithme discret, classique ou sur courbe elliptique (ECDLP).
- Le padding (OAEP pour le chiffrement, PSS pour la signature) n'est pas optionnel : RSA brut est déterministe et vulnérable à des attaques structurelles connues, quelle que soit la taille de la clé.
- Diffie-Hellman pur n'authentifie pas les parties et reste vulnérable à une attaque de l'homme du milieu ; il doit être combiné à un mécanisme d'authentification (certificats, signatures) pour être utilisable en pratique.
- ECC atteint un niveau de sécurité équivalent à RSA avec des clés bien plus courtes (256 bits ECC $\approx$ 3072 bits RSA), un avantage décisif en performance et en empreinte réseau.

Chapitre 5 — Signatures numériques et infrastructures à clés publiques (PKI)

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe de la signature numérique

La signature numérique fournit simultanément trois propriétés que ni le chiffrement symétrique ni un simple MAC (chapitre 3) ne peuvent apporter ensemble : **authenticité** (le signataire est identifié), **intégrité** (le message n'a pas été altéré depuis la signature) et surtout **non-répudiation** (le signataire ne peut pas nier ultérieurement, y compris devant un tiers, être l'auteur de la signature). Le signataire utilise sa clé **privée** pour produire une signature liée de façon unique au message ; quiconque possède la clé **publique** correspondante peut vérifier cette signature, sans jamais pouvoir en produire une valide lui-même, puisqu'il ne détient pas la clé privée.

Schéma général

1. Calculer l'empreinte du message : $h = H(\text{message})$, à l'aide d'une fonction de hachage cryptographique (chapitre 3), typiquement SHA-256 ou SHA-512.
2. Signer l'empreinte avec la clé privée : $\text{signature} = \text{Sign}(\text{clé privée}, h)$.
3. Vérification : le vérificateur recalcule indépendamment $H(\text{message})$ à partir du message reçu, puis vérifie la signature reçue avec la clé publique du signataire présumé (via $\text{Verify}(\text{clé publique}, h, \text{signature})$).

Signer l'empreinte du message plutôt que le message entier apporte deux avantages pratiques essentiels. D'une part, cela permet de traiter des messages de taille totalement arbitraire (un document de quelques octets ou un fichier de plusieurs gigaoctets) avec des opérations cryptographiques de taille fixe, puisque l'empreinte a toujours la même longueur quelle que soit la taille du message d'origine. D'autre part, cela rend toute modification, même minime, du message détectable : la moindre altération change complètement l'empreinte calculée (effet avalanche du hachage), ce qui invalide immédiatement la signature lors de la vérification.

2. RSA-PSS

RSA-PSS est un schéma de signature basé sur RSA, utilisant un padding **probabiliste** (PSS — *Probabilistic Signature Scheme*), aujourd'hui recommandé pour toute nouvelle mise en œuvre de signature RSA. Le caractère probabiliste signifie qu'un même message signé deux fois avec la même clé privée produit deux signatures différentes (grâce à un sel aléatoire intégré au calcul), contrairement à un schéma déterministe.

Ce schéma est aujourd'hui privilégié par rapport à l'ancien standard **PKCS#1 v1.5**, un padding déterministe pour la signature, encore largement présent dans des systèmes existants pour des raisons de compatibilité mais moins recommandé pour de nouveaux déploiements en raison de propriétés de sécurité plus fragiles face à certaines analyses formelles, même si aucune attaque pratique généralisée ne le rend inutilisable dans tous les contextes. La recommandation générale reste néanmoins de préférer PSS dès que la compatibilité le permet.

3. ECDSA et Ed25519

Deux schémas de signature fondés sur la cryptographie sur courbes elliptiques (chapitre 4) dominent les déploiements modernes, avec des propriétés de robustesse sensiblement différentes :

- **ECDSA** (*Elliptic Curve Digital Signature Algorithm*) : largement déployée (certificats TLS, Bitcoin et de nombreuses blockchains, cartes à puce). Point de vigilance majeur : la sécurité d'ECDSA dépend crucialement de la génération, à **chaque** signature, d'un nombre aléatoire (le « nonce ») qui doit être à la fois unique et totalement imprévisible. Un nonce réutilisé entre deux signatures différentes, ou même simplement prévisible (généré par un générateur aléatoire de mauvaise qualité), permet à un attaquant de retrouver directement la clé privée du signataire par un calcul algébrique simple — une classe de faille qui a été illustrée par des compromissions réelles documentées, notamment sur certaines consoles de jeu et des portefeuilles Bitcoin dont l'implémentation générait des nonces de façon défailante.
- **Ed25519** : schéma de signature moderne fondé sur la courbe Curve25519 (chapitre 4), conçu pour éliminer structurellement le risque décrit ci-dessus : il est **déterministe**, c'est-à-dire que le nonce nécessaire au calcul est dérivé de façon reproductible de la clé privée et du message lui-même, plutôt que tiré d'un générateur aléatoire externe à chaque signature. Cette conception élimine par construction le risque de nonce faible ou réutilisé. Ed25519 est en outre notablement plus rapide qu'ECDSA à niveau de sécurité comparable, et largement recommandé pour les nouveaux déploiements (SSH, de nombreux protocoles et bibliothèques cryptographiques modernes).

4. Certificats numériques

Un certificat numérique établit un lien vérifiable entre une **identité** (une personne, un serveur, une organisation) et une **clé publique** donnée, ce lien étant lui-même garanti par la signature numérique d'une **autorité de certification** (AC, ou *CA* — *Certificate Authority*) reconnue et de confiance. Le format standard le plus répandu pour structurer ces certificats est **X.509**.

Contenu typique d'un certificat X.509

- L'identité du sujet (par exemple le nom de domaine complet pour un certificat de serveur web, ou le nom d'une organisation).
- La clé publique appartenant au sujet.
- L'identité de l'autorité de certification qui a émis et signé le certificat.
- La période de validité du certificat (date de début et date d'expiration).
- La signature numérique de l'AC, calculée sur l'ensemble des informations précédentes, qui garantit que ces informations n'ont pas été modifiées depuis l'émission.

Un certificat n'est donc, dans son essence, qu'une déclaration signée : « l'AC atteste que cette clé publique appartient bien à cette identité ». Toute la confiance accordée au certificat repose in fine sur la confiance accordée à l'AC émettrice.

5. Infrastructure à clés publiques (PKI)

Une PKI (*Public Key Infrastructure*) désigne l'ensemble des composants techniques et des procédures organisationnelles permettant de créer, distribuer, vérifier et révoquer des certificats à grande échelle :

- **Autorité de certification (AC/CA)** : l'entité qui émet et signe effectivement les certificats, engageant sa réputation (et souvent sa responsabilité contractuelle) sur la validité des identités qu'elle certifie.
- **Autorité d'enregistrement (RA — *Registration Authority*)** : l'entité chargée de vérifier l'identité du demandeur avant l'émission d'un certificat par l'AC — une étape souvent déléguée, distincte de l'AC elle-même, dont la rigueur conditionne directement la fiabilité de tous les certificats émis en aval.
- **Chaîne de confiance** : dans la plupart des déploiements, un certificat de serveur est signé non pas directement par une AC racine, mais par un certificat **intermédiaire**, lui-même signé par l'AC racine. Cette hiérarchie permet de limiter l'exposition de la clé privée de l'AC racine (conservée hors ligne dans des conditions de sécurité renforcées) tout en permettant l'émission courante de certificats via les intermédiaires. La clé publique de l'AC racine constitue l'**ancrage de confiance** (*trust anchor*), préinstallée dans les navigateurs et les systèmes d'exploitation, à partir de laquelle toute la chaîne est vérifiée.
- **Révocation** : un certificat peut devoir être invalidé avant sa date d'expiration normale (clé privée compromise, changement d'identité du sujet). Deux mécanismes coexistent : les **listes de révocation de certificats** (CRL — listes téléchargées périodiquement, listant les certificats révoqués) et **OCSP** (*Online Certificate Status Protocol* — vérification en ligne, en temps réel, du statut d'un certificat précis).

Vérification d'un certificat côté client

Lorsqu'un client (navigateur, application) reçoit un certificat, il doit effectuer plusieurs vérifications indépendantes, **toutes** nécessaires pour que la protection soit effective :

1. Le certificat est-il signé par une AC de confiance, la chaîne de signatures remontant bien jusqu'à une ancre de confiance reconnue ?
2. Le certificat est-il dans sa période de validité (ni pas encore valide, ni expiré) ?
3. Le certificat correspond-il bien à l'identité attendue par le client (par exemple, le nom de domaine visité correspond-il au sujet du certificat) ?
4. Le certificat a-t-il été révoqué depuis son émission (vérification CRL ou OCSP) ?

Une faille dans **n'importe laquelle** de ces vérifications — accepter un certificat expiré, ne pas vérifier que le nom de domaine correspond effectivement, ignorer le statut de révocation — annule intégralement la protection apportée par toute la chaîne cryptographique en amont, aussi robuste soit-elle mathématiquement. C'est un point classique d'audit de sécurité applicative (module *Audit organisation et technique*, chapitre 5) : la cryptographie peut être parfaitement correcte tout en étant rendue inefficace par une vérification incomplète côté client.

6. Modèles alternatifs

Le modèle hiérarchique de PKI décrit ci-dessus n'est pas le seul modèle de confiance existant en cryptographie à clé publique :

- **Web of Trust** (utilisé historiquement par PGP/GPG) : modèle de confiance **distribuée** entre pairs, sans autorité centrale unique. Chaque utilisateur peut signer la clé publique d'un autre utilisateur pour attester qu'il a vérifié son identité par un moyen indépendant ; la confiance globale émerge du réseau de signatures croisées ainsi constitué, plutôt que d'une hiérarchie descendante. Ce modèle évite la dépendance à une AC unique, mais peine à s'imposer à grande échelle en dehors de communautés techniques restreintes, notamment en raison de sa complexité d'usage pour un utilisateur non spécialiste.
- **Certificate Transparency** : registres publics, append-only et vérifiables cryptographiquement (fondés sur des arbres de Merkle, chapitre 3), dans lesquels toute AC participante doit journaliser chaque certificat qu'elle émet. Ce mécanisme permet de détecter, y compris a posteriori, l'émission frauduleuse ou négligente d'un certificat par une AC compromise ou défaillante, en rendant l'ensemble des certificats émis publiquement auditables par quiconque, y compris le titulaire légitime d'un domaine qui souhaite vérifier qu'aucun certificat non autorisé n'a été émis en son nom.

À retenir

- La signature numérique apporte authenticité, intégrité et non-répudiation ; la non-répudiation, en particulier, est absente d'un simple MAC (chapitre 3).
- ECDSA exige un nonce aléatoire de haute qualité, unique à chaque signature, faute de quoi la clé privée peut être retrouvée ; Ed25519 élimine ce risque par une construction déterministe.
- Un certificat X.509 lie une identité à une clé publique via la signature d'une AC ; une PKI repose sur une chaîne de confiance hiérarchique jusqu'à une ancre de confiance préinstallée.
- La robustesse d'une PKI dépend entièrement de la rigueur de vérification à chaque maillon (identité, période de validité, correspondance du domaine, révocation) — une seule vérification omise annule la protection de toute la chaîne.

Chapitre 6 — Protocoles appliqués : TLS et cryptographie post-quantique

Cryptographie

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. TLS (Transport Layer Security) : rôle et positionnement

TLS sécurise les communications entre un client et un serveur — l'exemple le plus visible étant HTTPS (HTTP sur TLS), mais aussi de nombreux autres protocoles applicatifs (messagerie, bases de données distantes, VPN applicatifs) — en fournissant trois garanties combinées : la **confidentialité** du trafic échangé, son **intégrité** (toute altération est détectée) et l'**authentification** du serveur (et, optionnellement mais plus rarement en pratique, du client également, via un certificat client). TLS n'est pas une nouvelle primitive cryptographique : c'est un **protocole d'orchestration** qui assemble, dans un ordre précis et standardisé, l'ensemble des briques cryptographiques étudiées dans les chapitres précédents de ce module — échange de clé asymétrique, certificats et signatures, chiffrement symétrique authentifié, fonctions de hachage. À ce titre, TLS constitue une synthèse pratique particulièrement instructive de tout ce qui précède.

2. La poignée de main TLS 1.3 (simplifiée)

La **poignée de main** (*handshake*) est la phase initiale d'une connexion TLS, durant laquelle client et serveur négocient les paramètres cryptographiques et établissent les clés de session, avant que le trafic applicatif proprement dit ne commence :

1. **ClientHello** : le client propose la liste des suites cryptographiques qu'il supporte, ainsi que des paramètres pour un échange de clé — notamment une clé publique éphémère pour un échange Diffie-Hellman sur courbe elliptique (chapitre 4), envoyée dès ce premier message pour accélérer la négociation par rapport aux versions antérieures de TLS.
2. **ServerHello** : le serveur choisit, parmi les propositions du client, la suite cryptographique retenue, complète l'échange de clé de son côté et transmet son **certificat** numérique.
3. **Vérification du certificat** : le client valide la chaîne de confiance présentée par le serveur jusqu'à une AC racine reconnue et préinstallée (ancrage de confiance, chapitre 5), et vérifie que le certificat correspond bien au nom de domaine visité, qu'il est dans sa période de validité, et qu'il n'a pas été révoqué.
4. **Dérivation des clés de session** : à partir du secret partagé obtenu par l'échange de clé Diffie-Hellman éphémère, une fonction de dérivation de clé (HKDF, fondée sur HMAC — chapitre 3) produit l'ensemble des clés symétriques distinctes utilisées pour le reste de la session (généralement une clé pour chaque sens de communication).
5. **Communication chiffrée** : l'ensemble du trafic applicatif qui suit est protégé par un chiffrement symétrique **authentifié** (mode AEAD — AES-GCM ou ChaCha20-Poly1305, chapitre 2), garantissant à la fois confidentialité et intégrité de chaque message échangé.

TLS 1.3, standardisé en 2018, a significativement simplifié cette poignée de main par rapport à TLS 1.2 (réduisant le nombre d'allers-retours réseau nécessaires avant que le trafic chiffré ne puisse commencer), et a surtout **supprimé du protocole** l'ensemble des suites cryptographiques jugées faibles ou obsolètes — le chiffrement RC4, les modes CBC insuffisamment protégés contre les attaques de padding oracle, et l'échange de clé via RSA statique (qui ne fournit pas de confidentialité persistante, voir section suivante). TLS 1.3 impose désormais, par défaut et sans possibilité de repli, un échange de clé Diffie-Hellman **éphémère**.

3. Confidentialité persistante (Perfect Forward Secrecy)

La **confidentialité persistante** (ou *Perfect Forward Secrecy*, PFS) est la propriété garantissant que la compromission ultérieure de la clé privée à long terme d'un serveur (celle associée à son certificat) ne permet **pas** de déchiffrer rétroactivement des sessions passées qui auraient été interceptées et enregistrées par un adversaire des années auparavant.

Cette propriété s'obtient en utilisant, pour chaque session, des clés d'échange **éphémères** — générées spécifiquement pour cette session, utilisées le temps de dériver les clés symétriques, puis immédiatement détruites — typiquement via un échange **Diffie-Hellman éphémère sur courbe elliptique** (noté **ECDHE**). Dans ce schéma, la clé privée à long terme du serveur (celle du certificat) ne sert **qu'à signer** l'échange de clé éphémère pour en authentifier l'origine, mais n'intervient jamais directement dans le calcul du secret partagé lui-même. Ainsi, même si cette clé privée à long terme venait à être compromise ultérieurement, l'attaquant ne pourrait pas reconstituer, a posteriori, les secrets éphémères de sessions déjà terminées et détruites. C'est précisément l'une des raisons pour lesquelles TLS 1.3 impose ce mécanisme par défaut, en abandonnant l'ancien échange par RSA statique qui ne l'offrait pas.

4. Suites cryptographiques et obsolescence

Une **suite cryptographique** (*cipher suite*) combine, en un seul identifiant négocié durant la poignée de main, un mécanisme d'échange de clé, un algorithme de signature pour l'authentification, un algorithme de chiffrement symétrique et une fonction de hachage utilisée pour la dérivation de clé et l'intégrité. Les recommandations concernant les suites jugées acceptables évoluent dans le temps, à mesure que la cryptanalyse progresse et que des faiblesses sont découvertes dans des algorithmes ou des tailles de clé auparavant considérés comme sûrs (rappel du module *Cryptanalyse*).

Plusieurs versions et composants sont aujourd'hui déconseillés, voire explicitement interdits, par les référentiels de sécurité de référence (comme les recommandations de l'ANSSI ou les exigences du standard PCI-DSS pour les environnements traitant des données de paiement) : les versions de protocole **SSL 2.0 et 3.0**, **TLS 1.0 et 1.1**, l'algorithme de chiffrement par flux **RC4**, ainsi que les suites historiques dites « **EXPORT** » (issues d'anciennes restrictions réglementaires américaines sur l'exportation de cryptographie forte, qui imposaient des clés artificiellement affaiblies). Le maintien de telles suites activées côté serveur, même en simple option de repli, constitue une non-conformité relevée systématiquement lors d'un audit de configuration TLS.

5. Erreurs d'implémentation TLS fréquentes (audit)

Au-delà du choix du protocole et des suites cryptographiques elles-mêmes, la sécurité effective d'un déploiement TLS dépend fortement de la rigueur de son implémentation côté client comme côté serveur. Les erreurs les plus fréquemment relevées lors d'un audit incluent :

- **Ne pas vérifier le nom de domaine du certificat** : accepter un certificat valide mais délivré pour un domaine différent de celui effectivement contacté, ce qui annule la protection contre l'usurpation de serveur.
- **Accepter des certificats auto-signés ou expirés en production** : une pratique parfois tolérée en développement pour simplifier les tests, mais qui ne doit jamais subsister en environnement de production, où elle prive le client de toute garantie d'authenticité du serveur.
- **Désactiver la vérification de certificat pendant le développement, avec un oubli de réactivation en production** : une erreur classique et récurrente, où un indicateur de configuration destiné à faciliter les tests locaux (par exemple avec des certificats auto-signés) reste actif après le déploiement, exposant l'application entière à une attaque de l'homme du milieu sans qu'aucune alerte ne soit levée.
- **Utiliser des suites cryptographiques obsolètes non désactivées côté serveur** : conserver, par défaut ou par excès de prudence vis-à-vis d'anciens clients, la possibilité de négocier des suites faibles ou dépréciées mentionnées à la section précédente.

Ces points recoupent directement la grille d'audit applicatif vue dans le module *Audit organisation et technique*, et illustrent une fois de plus le constat déjà rencontré au chapitre 5 : une cryptographie mathématiquement solide ne protège rien si son implémentation ou sa configuration comporte une faille.

6. La menace post-quantique et la migration en cours

L'**algorithme de Shor** (rappel : module *Cryptanalyse*, chapitre 5), s'il était exécuté sur un ordinateur quantique disposant d'un nombre suffisant de qubits stables et corrigés d'erreurs, permettrait de résoudre efficacement à la fois le problème de factorisation (cassant RSA) et le problème du logarithme discret, classique ou sur courbe elliptique (cassant Diffie-Hellman et ECC). Un tel ordinateur, à l'échelle nécessaire pour menacer les tailles de clé utilisées aujourd'hui, n'est pas disponible à ce jour. Deux considérations motivent néanmoins une action de migration dès maintenant, plutôt que d'attendre qu'un tel ordinateur existe réellement :

- « **Harvest now, decrypt later** » (« récolter maintenant, déchiffrer plus tard ») : un adversaire disposant de capacités d'interception peut enregistrer dès aujourd'hui du trafic chiffré, dans l'espoir de le déchiffrer ultérieurement une fois l'informatique quantique devenue suffisamment mature. Cette stratégie représente un risque réel et concret pour toute donnée dont la **confidentialité doit être préservée sur une longue durée** (secrets d'État, données médicales, secrets industriels de long terme), même si le déchiffrement effectif n'interviendrait que dans plusieurs années.
- **Délai de migration** : le déploiement d'une nouvelle famille cryptographique à l'échelle d'Internet — mise à jour des bibliothèques, des protocoles, des équipements matériels, des certificats — prend typiquement plusieurs années, ce qui justifie d'anticiper la transition bien avant qu'elle ne devienne urgente, plutôt que d'attendre l'apparition effective de la menace.

Le NIST a standardisé en 2024 un ensemble d'algorithmes post-quantiques issus de son concours international : **ML-KEM** (anciennement connu sous le nom **Kyber** durant le processus de standardisation), destiné à l'échange de clé, fondé sur la difficulté de problèmes issus des réseaux euclidiens (*lattice-based cryptography*) plutôt que sur la factorisation ou le logarithme discret ; et **ML-DSA** (anciennement **Dilithium**), destiné à la signature numérique, fondé sur une famille de problèmes mathématiques similaire. L'intégration progressive de ces algorithmes dans TLS et d'autres protocoles est actuellement en cours, le plus souvent sous forme **hybride** : l'échange de clé combine simultanément un mécanisme classique (par exemple ECDHE) et un mécanisme post-quantique (par exemple ML-KEM), la sécurité globale de l'échange dépendant alors de la robustesse d'au moins l'un des deux mécanismes. Cette approche hybride reflète une prudence délibérée : elle évite de dépendre uniquement de la maturité encore relativement récente de ces nouveaux algorithmes, dont la résistance à long terme à la cryptanalyse n'a pas encore été éprouvée sur la même durée que RSA ou ECC.

À retenir

- TLS combine, de façon opérationnelle et standardisée, échange de clé asymétrique éphémère, authentification par certificat et chiffrement symétrique authentifié — la synthèse pratique de tout ce module.
- La confidentialité persistante (clés éphémères, ECDHE) protège les échanges passés même en cas de compromission future d'une clé privée long terme du serveur.
- Une implémentation ou une configuration TLS défaillante (vérification de certificat désactivée, suites obsolètes maintenues) annule la protection apportée par une cryptographie par ailleurs solide.
- La transition post-quantique (ML-KEM, ML-DSA) est engagée par anticipation, notamment face au risque « harvest now, decrypt later » sur des données sensibles à confidentialité de longue durée, et déployée le plus souvent sous forme hybride avec les mécanismes classiques.